

INFORME PRELIMINAR - DESAFÍO 1

Implementación Sweet Crush mediante manipulación de bits en C++

INFORMÁTICA 2

JULIANA MENDEZ MOSQUERA

YEIMMY YISSEL CUESTA JIMÉNEZ

Contextualización:

El desafío #1 consiste en desarrollar e implementar un programa del juego Sweet Crush, empleando conceptos a nivel de Bits y gestión de memoria dinámica, bajo una estructura de programación estricta. El objetivo principal es crear un tablero dinámico capaz de eliminar fichas de manera individual, detectar combinaciones, modificar estructuras y gestionar diversas situaciones durante el juego. El tablero incluirá seis tipos de fichas representadas a nivel de bits (3), almacenadas en bytes, siguiendo una secuencia continua y compacta de bits.

El diseño del tablero permitirá cambiar su tamaño, facilitando al usuario la adición y eliminación de filas y columnas de manera intermedia y aleatoria. La implementación de memoria dinámica es crucial, debiendo determinar cuánta memoria necesita el tablero. La modificación física de la memoria deberá llevarse a cabo cuando la utilización caiga por debajo del 65 % respecto al tamaño actual, especialmente tras la eliminación de filas o columnas.

El programa también gestionará el desplazamiento vertical, llenando espacios vacíos con nuevas fichas aleatorias, y ejecutará una secuencia de cascadas en la parte superior del tablero que incluye: detectar combinaciones, eliminar fichas, reorganizar el tablero, agregar nuevas y revisar movimientos.

Análisis:

Crear un tablero rectangular dinámico.

- El tablero debe ser dimensionado por X Filas y Columnas definidas por el usuario al iniciar.
- Siendo las filas y columnas variables cambiantes no constantes.
- Si se modifican las filas o columnas, la memoria debe reorganizarse.
- Las filas y columnas deben ocuparse con fichas de manera aleatoria y mostradas en el tablero.

Manejar 6 tipos de fichas.

- 6 tipos diferentes de fichas diferenciadas por (figuras geométricas) individualmente identificables y fáciles de distinguir para el usuario.
- entendiendo que cada ficha equivale a 3 bits permitiendo un total de 8 combinaciones:

Representar cada ficha por 3 bits.

- Cada 1 byte nos permite almacenar 8 bits, es decir, al tener 3 fichas sucesivas la 3ra fragmentará sus 3 bits de manera que los dos primeros, hacen parte de un primer byte y su tercer bit se almacena en un segundo byte.

Aplicando la fórmula: **Memoria = posición (i) * bit necesarios (3) / 8**

- Una vez identificado el byte, necesitamos saber en qué posición de bits dentro de ese byte comienza la ficha.

Entendiendo que cada bit entrante, ocupará el primer espacio disponible, siendo el bit 7 el primero en ser ocupado y el bit 0 el último.

Aplicando la fórmula: **Desplazamiento = posición (i) * bit necesarios (3) Mod 8**

Memoria total=

cantidad de posiciones = Filas * Columnas.

bit totales = posiciones * 3

Almacenar todo el tablero en una secuencia continua y compacta de bits

- Al manipular cada una de las fichas se busca empaquetar todas las fichas linealmente

Utilizar punteros y memoria dinámica.

Permitir eliminar fichas.

- El usuario debe poder eliminar una posición y esa ficha debe desaparecer.
- No se elimina la memoria física ni el espacio, más bien, se sobrescriben los 3 bits de dicha posición con el valor definido como espacio vacío o **110** en su codificación binaria.

Detectar combinación de 3 o más fichas iguales horizontal a vertical.

- Se debe recorrer el tablero usando ciclos anidados para cada fila y columnas
- El ciclo debe comparar que los 3 bits de una ficha coinciden con una ficha adyacente. Se sumarían a un contador.
- Si el contador llega a 3 o más fichas iguales linealmente, se debe guardar esas posiciones para el siguiente paso.

Eliminar combinaciones simultáneas correctamente.

- Si al formarse una combinación, en la cual una misma ficha hace parte de una línea horizontal y vertical simultáneamente, no se puede borrar a medias ya que impediría la validación del otro eje.
- Estratégicamente se debe marcar todas las posiciones que formarán parte de un movimiento y al mismo tiempo cambiar esos bits a espacios vacíos.

Generar nuevas fichas.

- Después de que las fichas caen por la gravedad, los espacios vacíos van a quedar acumulados en la parte superior del tablero.
- En este punto debemos usar una función de aleatoriedad para generar números entre 0 y 5 que corresponden a las combinaciones de 000 a 101, correspondientes a las fichas disponibles.
- Luego sobrescribimos esos bits en la memoria para rellenar los espacios.

Detectar y procesar cascadas.

- Al caer fichas nuevas, se pueden formar líneas de 3 o más por hecho de casualidad.
- En estos casos, debemos de incluir las funciones de guardar, eliminar, completar y generar fichas dentro de un ciclo mientras que se ejecute hasta confirmar que no hay más líneas iguales en el tablero.

Agregar y eliminar filas / Agregar y eliminar columnas.

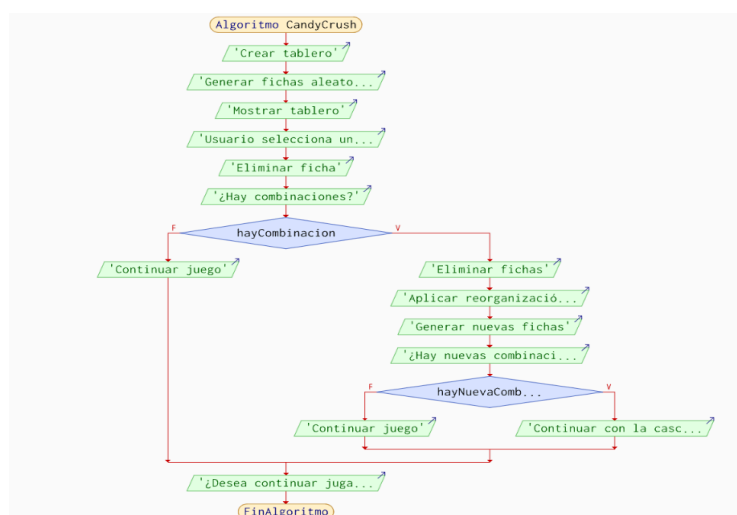
- Al modificar las filas o columnas, cambian las dimensiones totales del tablero, por lo que toca aplicar el manejo de memoria dinámica.
- Al agregar o quitar columnas, los bits se desordenan respecto al tablero anterior, así que hay que recalcular el empaquetamiento ficha por ficha.

Modificar directamente la memoria cuando corresponda.

- Todo el procesamiento de los puntos anteriores se tiene que lograr manipulando los bytes con punteros y operadores lógicos a nivel de bits.

Diseño:

Figura 1: Planteamiento general del funcionamiento, como estructura del juego.



Descomposición funcional del sistema y funciones:

- **Dimensiones tablero;**
 - Arreglo del tablero
 - CrearTablero()
 - Insertar filas y columnas
 - Eliminar filas y columnas
 - Limpiar tablero
 - Convertir filas y columnas en posición.
 - posiciones libres
 - Mostrar representación binaria.
 - **Almacenamiento de Fichas:**
 - CalcularBytes()
 - Calcularbits()
 - extraerfichas()
 - guardarfichas()
 - Obtenerfichas()
 - modificarfichas()
 - **Diseño de fichas:** Fichas definidas por figuras geométricas.
 - Ficha 1: Cuadrado 000 -
 - Ficha 2: hexágono 001 -
 - Ficha 3: Rectángulo 010 -
 - Ficha 4: Triángulo 011 -
 - Ficha 5: Rombo 100 -
 - Ficha 6: Pentagono 101 -
 - Estado libre: círculo 110 -
 - Estado especial: estrella 111 -
 - **Buscar combinaciones:**
 - Horizontales - Filas (3 o más fichas)
 - Verticales - Columnas (3 o más fichas)
 - Ambas combinaciones fila * columna si coinciden.
 - **Registrar posiciones.**
 - **Eliminación de combinaciones:**
 - fila (4 o más fichas)
 - Columna (4 o más fichas)
 - Cruz
 - Actualizar arreglo.
 - **Gravedad :**
 - Desplazar fichas
 - reorganizar arreglo
 - sobre las columnas, las fichas de los espacios vacíos deberán desplazarse hacia abajo. Y moverse hacia la izquierda de manera vertical.
 - **Aplicación de la función cascada:**
 - búsqueda de combinación
 - Eliminación
 - Reorganización del tablero (gravedad)
 - generación de nuevas fichas
 - búsqueda de nuevas combinaciones
 - Repetición del proceso.
 - **Estadísticas:**
 - número de movimientos.
 - número de fichas eliminadas
 - número de combinaciones encontradas.
 - número de cascadas realizadas.
 - puntuación.
- Estadísticas:**
- void actualizarPuntuacion()
 - inicializarEstadisticas()
 - actualizarEstadisticas()
 - registrarMovimiento()
 - registrarCombinacion()
 - registrarFichaEliminada()
 - registrarCascada()
 - mostrarEstadisticas()
- Punteros + almacenamiento de 3 bits :**
- ObtenerPunteroByte()
 - ObtenerPosicionBit()
 - ExtraerFichaConPuntero()
 - InsertarFichaConPuntero()
 - ModificarFichaConPuntero()